# Integración de Servicios Cloud GCP

## Balanceo de carga, CDN y contenedores

**Semana 13 - Sesión 26**

Rel Guzman



# Revisión: lo que vimos en la Sesión 25

**Supervisar**

Mirar métricas y registros, y recibir una alerta cuando algo sale de lo normal.

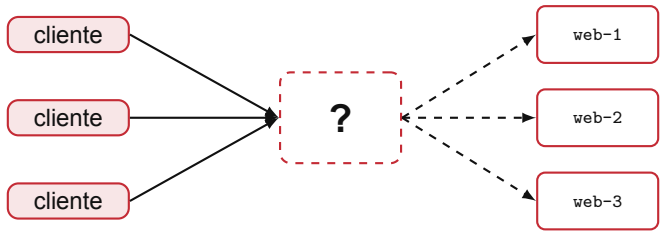
**Alta disponibilidad**

Tener varias copias en zonas distintas, para que una falla no apague el servicio.

**Responde**

1. ¿Qué servicio de Google Cloud te envía un correo si la CPU de `web-1` pasa del 80 %?
2. ¿Qué es un punto único de falla?
3. ¿Por qué conviene tener VMs en zonas distintas?

# Revisión: la pregunta de hoy




| 1. Balanceo de carga | 2. CDN | 3. Contenedores |
|---|---|---|
| Tenemos 3 servidores. ¿Cómo se reparten los clientes? | ¿Cómo hacer que las fotos carguen rápido en Lima? | ¿Cómo poner el mismo programa en los 3 servidores sin instalar todo a mano? |

**Logro de la sesión**

Al finalizar, podrás explicar con tus palabras qué es un **balanceador de carga**, qué es una **CDN** y para qué sirven **Docker**, **Docker Compose** y **Kubernetes**. Además, verás en tu computadora cómo Nginx reparte clientes entre tres servidores.

# Saberes previos

- Si 100 personas abren a la vez una página que corre en un solo servidor, ¿qué crees que pasa?
- Desde Lima abres la página de una tienda de EE. UU. ¿Crees que carga igual de rápido que una tienda con servidores en Lima?
- Un compañero te pasa su proyecto y en tu computadora no funciona. ¿Qué podría faltar?

# Utilidad de esta sesión

Cuando una aplicación tiene muchos clientes, se ponen **varias copias** de ella. Hoy verás cómo se reparten los clientes entre esas copias, cómo se entregan las fotos desde cerca y cómo se encienden copias iguales con un solo comando.

# Parte 1: balanceo de carga

Repartir a los clientes entre varios servidores

# El problema: un solo servidor

**El caso**

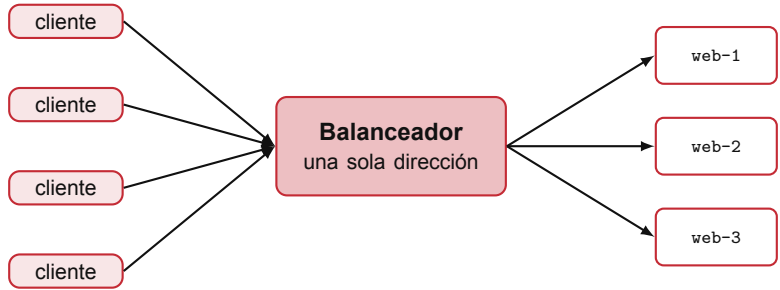
Seis clientes piden un reporte de ventas al mismo tiempo. Cada reporte tarda **5 segundos** y el servidor hace **uno a la vez**.




**Se forma una cola**

El cliente 6 espera **30 segundos**. ¿Y si hubiera 3 servidores?

# La solución: el balanceador de carga




**Balanceador de carga**

Recibe a todos los clientes en **una sola dirección** y los **reparte** entre varios servidores iguales.

**El cliente**

Solo conoce una dirección. No sabe cuántos servidores hay detrás.

**Dónde lo verás**

En Google Cloud: **Cloud Load Balancing**. En la práctica de hoy: **Nginx**.

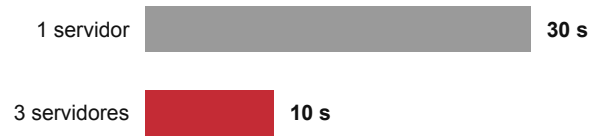
# Round robin: repartir por turnos

**Round robin**

Un cliente para cada servidor, en orden, y se vuelve a empezar.

| Cliente | Va a |
|---|---|
| 1 | `web-1` |
| 2 | `web-2` |
| 3 | `web-3` |
| 4 | `web-1` (otra vuelta) |
| 5 | `web-2` |
| 6 | `web-3` |

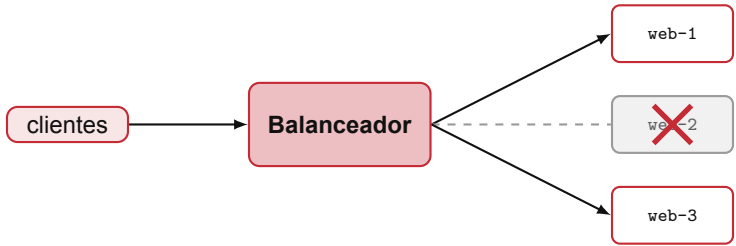
**Los 6 reportes de antes**




**Por qué**

Cada servidor hace 2 reportes, y los tres trabajan **al mismo tiempo**.

# ¿Y si un servidor se cae?




**Verificación de estado**

Cada pocos segundos, el balanceador le pregunta a cada servidor si responde. Si uno no responde, **deja de enviarle clientes** y reparte entre los que quedan.

**Lo que ve el cliente**

Nada raro: la página carga igual. Cuando `web-2` vuelve, el balanceador lo usa otra vez.

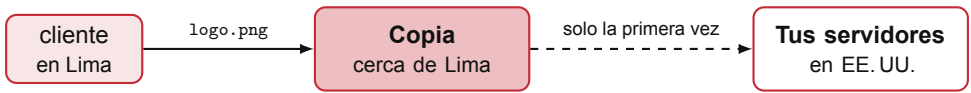
# Parte 2: CDN

Copias de los archivos cerca del cliente

# ¿Qué es una CDN?

**CDN (red de entrega de contenido)**

Servidores en muchas ciudades del mundo que guardan una **copia** de tus archivos (fotos, logo) y se los entregan al cliente **desde cerca**.




**La primera vez**

La CDN no tiene el logo: lo pide a tus servidores, se lo entrega al cliente y **guarda una copia**.

**Las siguientes veces**

La CDN entrega su copia. Es más rápido y tus servidores **ni se enteran**.

# ¿Qué va en la CDN?

| Sí va en la CDN | No va en la CDN |
|---|---|
| Logo | El carrito de Ana |
| Fotos de productos | Los pedidos de Luis |
| Archivos CSS y JavaScript | El estado de un pago |

**La regla**

Va lo que es **igual para todos**. No va lo que **cambia según el cliente**: Luis podría recibir la copia del carrito de Ana.

**En Google Cloud**

Se llama **Cloud CDN** y se activa con una casilla en el balanceador de carga.

# Parte 3: contenedores

Docker, Docker Compose y Kubernetes

# El problema: preparar cada servidor

**Para la práctica de hoy necesitas**

1. Instalar Node.js.
2. Instalar Nginx, que se instala distinto en Windows, macOS y Linux.
3. Abrir cinco terminales.

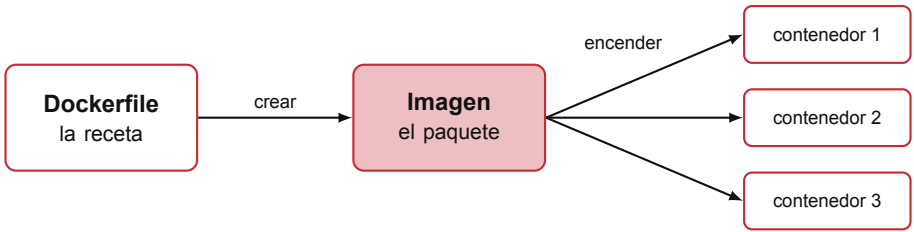
**Ahora imagina**

Hacerlo en 3 servidores. Si en uno instalas otra versión de Node.js, ese servidor podría fallar.

**Contenedor (lo viste en la Sesión 12)**

Un paquete con tu programa **y todo lo que necesita** para funcionar, para que corra igual en cualquier máquina.

# Docker: imagen y contenedor




| Imagen | Contenedor |
| --- | --- |
| El paquete guardado: el programa y todo lo que necesita. Está apagado. | Una imagen **encendida**. De una imagen puedes encender muchos contenedores iguales. |

**Docker** es el programa que crea las imágenes y enciende los contenedores.

# El Dockerfile de la tienda

```
FROM node:22-alpine
WORKDIR /app
COPY package.json server.js ./
CMD ["node", "server.js", "3001"]
```

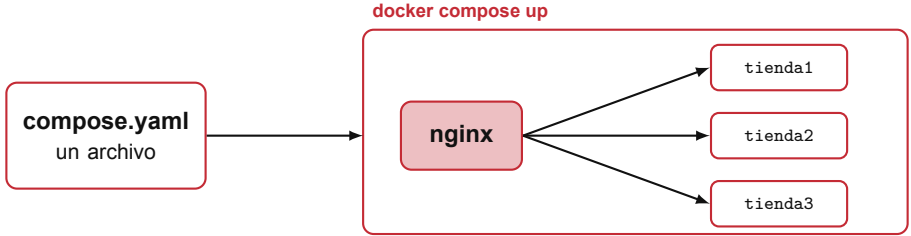
| Línea | En palabras |
|---|---|
| `FROM` | Empieza con Node.js ya instalado |
| `WORKDIR` | Trabaja en la carpeta `/app` |
| `COPY` | Copia los archivos de la tienda |
| `CMD` | Al encender, ejecuta la tienda |

```
docker build -t tienda .          # crea la imagen "tienda"
docker run -p 3001:3001 tienda    # enciende un contenedor
```

**Lo importante**

No instalaste Node.js en tu computadora: viene dentro de la imagen.

# Docker Compose: varios contenedores a la vez

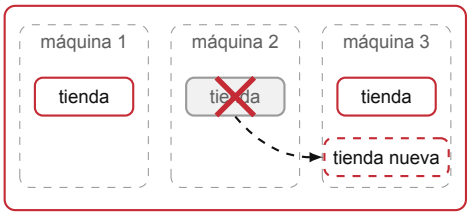



## Docker Compose

Describe en **un archivo** todos los contenedores de tu aplicación y los enciende juntos con **un comando**.

| A mano (Práctica 1) | Con Docker Compose (Práctica 2) |
|---|---|
| Instalar Node.js y Nginx | Solo tener Docker |
| 5 terminales, 5 comandos | 1 comando: `docker compose up` |

# Kubernetes: contenedores en muchas máquinas




**¿Por qué no basta Compose?**

Compose usa **una sola máquina**. Si esa máquina se apaga, se apaga todo.

**Kubernetes**

Tú le dices: «quiero **3 copias** de la tienda». Kubernetes las reparte en varias máquinas y se encarga de que **siempre haya 3**.

**Si se cae una copia**

Kubernetes lo nota y enciende otra, sin que tú hagas nada.

**En Google Cloud**

Se llama **Google Kubernetes Engine** (GKE).

# Las tres herramientas

| | Docker | Docker Compose | Kubernetes |
|---|---|---|---|
| **Qué hace** | Crea la imagen y enciende un contenedor | Enciende varios contenedores juntos | Mantiene muchas copias encendidas |
| **Dónde** | Una máquina | Una máquina | Varias máquinas |
| **Ejemplo** | La imagen de la tienda | Práctica 2: 3 tiendas + Nginx | La tienda en producción |

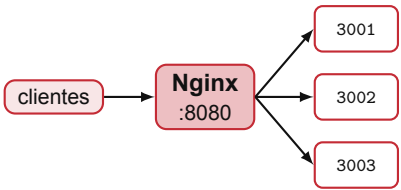
**En una frase**

Docker **empaqueta**, Docker Compose **junta** y Kubernetes **mantiene**.

# Práctica 1: balanceo con Nginx en tu computadora

## Caso de estudio

Pedidos Express prepara su tienda online para el Cyber Day: tres copias del servidor (Node.js) detrás de **Nginx**, que reparte por turnos: 3001, 3002, 3003, 3001… (*round robin*).




| Ruta | Qué hace | El servidor… |
|---|---|---|
| `GET /` | Página principal | responde al instante |
| `GET /productos` | Consulta la base de datos | **espera** 2 s |
| `POST /comprar` | Cobra con el banco | **espera** 5 s |
| `GET /reporte` | Calcula un reporte | **trabaja** 5 s con la CPU |

**Cómo empezar (carpeta `Sesion26/practica_balanceo/`)**

1. Tres terminales, un servidor en cada una: `node server.js 3001` (y 3002, 3003)
2. Otra terminal, para encender Nginx: `nginx -p "$PWD" -c nginx.conf`
3. Otra terminal, para simular clientes: `node prueba.js / 6 --en-fila`

# Práctica 1: seis pruebas

| # | Comando | Qué observar |
|---|---|---|
| 1 | `node prueba.js / 6 --en-fila` | ¿En qué orden responden los servidores? |
| 2 | `node prueba.js /productos 6 --directo 3001` | Un solo servidor: ¿cuánto tarda en total? |
| 3 | `node prueba.js /reporte 6 --directo 3001` | Un solo servidor: ¿cuánto espera el sexto cliente? |
| 4 | `node prueba.js /reporte 6` | Con Nginx: ¿cuánto tarda en total? |
| 5 | `node prueba.js /reporte 1`<br>y, antes de 5 s, en otra terminal:<br>`node prueba.js / 6` | ¿Qué visitas tardan? ¿A qué servidor le tocaron? |
| 6 | Apaga el 3002 (`Ctrl+C`) y luego:<br>`node prueba.js / 6 --en-fila` | ¿Algún cliente ve un error? ¿Qué servidores responden? |

**La idea**

Cuando el servidor **espera**, atiende a muchos a la vez. Cuando **trabaja** con la CPU, los clientes hacen cola: ahí ayuda repartir entre varios servidores.

# Práctica 2: la misma tienda con Docker Compose

**Antes de empezar**

Apaga los servidores y Nginx de la Práctica 1. Necesitas **Docker Desktop**. Si no lo tienes, sigue la demostración del docente.

```
docker compose up --build -d
docker compose ps
node prueba.js / 6 --en-fila
docker compose down
```

**Qué observar**

1. ¿Cuántos contenedores muestra `docker compose ps`?
2. ¿La prueba da lo mismo que en la Práctica 1?
3. ¿Cuántos comandos usaste para encender todo? ¿Y en la Práctica 1?

Los archivos `Dockerfile` y `compose.yaml` están en la misma carpeta. La guía completa está en el `README.md`.

# Cierre: preguntas para comprobar

**Responde con tus palabras**

1. ¿Qué hace un balanceador de carga?
2. Con 3 servidores y round robin, ¿a cuál va el cliente 4?
3. Da un ejemplo de archivo que sí va en una CDN y uno que no.
4. ¿Qué diferencia hay entre una imagen y un contenedor?
5. ¿Qué hace Kubernetes que Docker Compose no hace?

**Idea final**

Muchos clientes se atienden con varias copias iguales: el balanceador las reparte, la CDN entrega las fotos desde cerca y los contenedores hacen que todas las copias sean idénticas.

# Referencias

- Prácticas: `github.com/rgap/rgap-Integracion-de-Servicios-Cloud-Gcp-Agosto2026`, carpeta `Sesion26/practica_balanceo/`.
- Nginx: `nginx.org`. Node.js: `nodejs.org`.
- Docker y Docker Compose: `docs.docker.com`. Kubernetes: `kubernetes.io`.
- Google Cloud: Cloud Load Balancing, Cloud CDN y Google Kubernetes Engine (GKE): `cloud.google.com`.
- Material previo del curso: Sesión 12 (contenedores) y Sesión 25 (supervisión y alta disponibilidad).
- Las respuestas de las preguntas están en `../respuestas.md`.